IMPORTANT QUESTION
0:5 INSTR
0:6,7 RTRIM, LTRIM
0:20, 0:21
0:28
0:34 MOST IMP
0:36, 37
0:45 46

Sukhvinder Singh

SQL Query Notes with Explanations

Q-1. Write an SQL query to fetch “FIRST_NAME” from Worker table using the alias name as <WORKER_NAME>.

```
SELECT first_name AS WORKER_NAME FROM worker;
```

Explanation:

Fetches the `first_name` column but renames it to `WORKER_NAME` in the output.

Q-2. Write an SQL query to fetch “FIRST_NAME” from Worker table in upper case.

```
SELECT UPPER(first_name) FROM worker;
```

Explanation:

Converts all characters in the `first_name` column to uppercase.

Q-3. Write an SQL query to fetch unique values of DEPARTMENT from Worker table.

```
SELECT DISTINCT department FROM worker;
```

Explanation:

Fetches unique department names from the table, removing duplicates.

Q-4. Write an SQL query to print the first three characters of FIRST_NAME from Worker table.

```
SELECT SUBSTRING(first_name, 1, 3) FROM worker;
```

Explanation:

Extracts the first three characters from the `first_name`.

Q-5. Write an SQL query to find the position of the alphabet (‘b’) in the first name column ‘Amitabh’ from Worker table.

```
SELECT INSTR(first_name, 'b') FROM worker WHERE first_name = 'Amitab
h';
```

Explanation: In Postgresql we use Position ('B' IN name)

Finds the position of the character `'b'` in the string `'Amitabh'`.

Q-6. Write an SQL query to print the FIRST_NAME from Worker table after removing white spaces from the right side.

```
SELECT RTRIM(first_name) FROM worker;
```

Explanation:

Removes trailing spaces from `first_name`.

Q-7. Write an SQL query to print the DEPARTMENT from Worker table after removing white spaces from the left side.

```
SELECT LTRIM(department) FROM worker
```

Explanation:

Removes leading spaces from the `department` field.

Note: Your original query had `first_name` instead of `department`.

Q-8. Write an SQL query that fetches the unique values of DEPARTMENT from Worker table and prints its length.

```
SELECT DISTINCT department, LENGTH(department) FROM worker;
```

Explanation:

Gets unique departments and their string lengths.

Q-9. Write an SQL query to print the FIRST_NAME from Worker table after replacing ‘a’ with ‘A’.

```
SELECT REPLACE(first_name, 'a', 'A') FROM worker;
```

Explanation:

Replaces all lowercase `'a'` with uppercase `'A'` in `first_name`.

Q-10. Write an SQL query to print the FIRST_NAME and LAST_NAME from Worker table into a single column COMPLETE_NAME. A space char should separate them.

```
sql
CopyEdit
SELECT CONCAT(first_name, ' ', last_name) AS COMPLETE_NAME FROM
worker;
```

Explanation:

Concatenates first and last names with a space in between.

Q-11. Write an SQL query to print all Worker details from the Worker table order by FIRST_NAME Ascending.

```
SELECT * FROM worker ORDER BY first_name ASC;
```

Explanation:

Orders all worker records by first name alphabetically.

Q-12. Write an SQL query to print all Worker details from the Worker table order by FIRST_NAME Ascending and DEPARTMENT Descending.

```
SELECT * FROM worker ORDER BY first_name ASC, department DESC;
```

Explanation:

Sorts workers by first name ascending, and if there are duplicates, orders those by department descending.

Q-13. Write an SQL query to print details for Workers with the first name as “Vipul” and “Satish” from Worker table.

```
SELECT * FROM worker WHERE first_name IN ('Vipul', 'Satish');
```

Explanation:

Fetches all workers whose first names are either ‘Vipul’ or ‘Satish’.

Q-14. Write an SQL query to print details of workers excluding first names, “Vipul” and “Satish” from Worker table.

```
SELECT * FROM worker WHERE first_name NOT IN ('Vipul', 'Satish');
```

Explanation:

Fetches all workers except those whose first names are ‘Vipul’ or ‘Satish’.

Q-15. Write an SQL query to print details of Workers with DEPARTMENT name as “Admin*”.

```
SELECT * FROM worker WHERE department LIKE 'Admin%';
```

Explanation:

Selects workers in departments starting with ‘Admin’ (like Admin, Administration, etc.).

Q-16. Write an SQL query to print details of the Workers whose FIRST_NAME contains ‘a’.

```
SELECT * FROM worker WHERE first_name LIKE '%a%';
```

Explanation:

Fetches workers whose first name contains the letter ‘a’ anywhere.

Q-17. Write an SQL query to print details of the Workers whose FIRST_NAME ends with ‘a’.

```
SELECT * FROM worker WHERE first_name LIKE '%a'
```

Explanation:

Fetches workers whose first name ends with the letter ‘a’.

Q-18. Write an SQL query to print details of the Workers whose FIRST_NAME ends with ‘h’ and contains six alphabets.

```
SELECT * FROM worker WHERE first_name LIKE '_____h';
```

Explanation:

Matches names exactly 6 characters long with last character ‘h’ (`_` represents one character).

Q-19. Write an SQL query to print details of the Workers whose SALARY lies between 100000 and 500000.

```
SELECT * FROM worker WHERE salary BETWEEN 100000 AND 500000;
```

Explanation:

Filters workers earning between 100,000 and 500,000 inclusive.

Q-20. Write an SQL query to print details of the Workers who have joined in Feb’2014.

```
SELECT * FROM worker WHERE YEAR(joining_date) = 2014 AND MONTH(j
oining_date) = 2;
```

Explanation:

Filters workers who joined specifically in February 2014.

Q-21. Write an SQL query to fetch the count of employees working in the department ‘Admin’.

```
SELECT department, COUNT(*) FROM worker WHERE department = 'Admi
n';
```

Explanation:

Counts workers in the ‘Admin’ department.

Q-22. Write an SQL query to fetch worker full names with salaries >= 50000 and <= 100000.

```
SELECT CONCAT(first_name, ' ', last_name) FROM worker WHERE salary B
ETWEEN 50000 AND 100000;
```

Explanation:

Gets full names of workers earning between 50,000 and 100,000.

Q-23. Write an SQL query to fetch the number of workers for each department in descending order.

```
SELECT department, COUNT(worker_id) AS no_of_worker FROM worker G
ROUP BY department ORDER BY no_of_worker DESC;
```

Explanation:

Groups workers by department and sorts departments by worker count descending.

Q-24. Write an SQL query to print details of the Workers who are also Managers.

```
SELECT w.* FROM worker AS w
INNER JOIN title AS t ON w.worker_id = t.worker_ref_id
WHERE t.worker_title = 'Manager';
```

Explanation:

Joins `worker` and `title` tables to find workers who have the title ‘Manager’.

Q-25. Write an SQL query to fetch number (more than 1) of same titles in the ORG of different types.

```
SELECT worker_title, COUNT(*) AS count FROM title GROUP BY worker_titl
e HAVING count > 1;
```

Explanation:

Finds worker titles that appear more than once.

Q-26. Write an SQL query to show only odd rows from a table.

```
SELECT * FROM worker WHERE MOD(worker_id, 2) <> 0;
```

Explanation:

Selects rows where `worker_id` is odd (remainder when divided by 2 is not zero).

Q-27. Write an SQL query to show only even rows from a table.

```
SELECT * FROM worker WHERE MOD(worker_id, 2) = 0;
```

Explanation:

Selects rows where `worker_id` is even.

Q-28. Write an SQL query to clone a new table from another table.

```
CREATE TABLE worker_clone LIKE worker;
INSERT INTO worker_clone SELECT * FROM worker;
SELECT * FROM worker_clone
```

Explanation:

Creates a copy of the table structure and copies all data.

Q-29. Write an SQL query to fetch intersecting records of two tables.

```
SELECT worker.* FROM worker
INNER JOIN worker_clone USING(worker_id);
```

Explanation:

Selects records existing in both `worker` and `worker_clone` based on `worker_id`.

Q-30. Write an SQL query to show records from one table that another table does not have.

```
SELECT worker.* FROM worker
LEFT JOIN worker_clone USING(worker_id)
WHERE worker_clone.worker_id IS NULL
```

Explanation:

Shows records in `worker` but not in `worker_clone`.

Q-31. Write an SQL query to show the current date and time.

```
SELECT CURDATE();
SELECT NOW()
```

Explanation:

`CURDATE()` returns current date; `NOW()` returns date and time.

Q-32. Write an SQL query to show the top n (say 5) records of a table ordered by descending salary.

```
SELECT * FROM worker ORDER BY salary DESC LIMIT 5;
```

Explanation:

Fetches top 5 earners.

Q-33. Write an SQL query to determine the nth (say n=5) highest salary from a table.

```
SELECT * FROM worker ORDER BY salary DESC LIMIT 4, 1
```

Explanation:

Skips the first 4 highest salaries and fetches the 5th highest.

Q-34. Write an SQL query to determine the 5th highest salary without using LIMIT keyword.

```
SELECT salary FROM worker w1
WHERE 4 = (
  SELECT COUNT(DISTINCT w2.salary)
  FROM worker w2
  WHERE w2.salary >= w1.salary
);
```

Explanation:

Finds salary where exactly 4 distinct salaries are greater or equal (5th highest).

Q-35. Write an SQL query to fetch the list of employees with the same salary.

```
SELECT w1.* FROM worker w1, worker w2
WHERE w1.salary = w2.salary AND w1.worker_id != w2.worker_id;
```

Explanation:

Finds workers sharing the same salary as another worker.

Q-36. Write an SQL query to show the second highest salary from a table using sub-query.

```
SELECT MAX(salary) FROM worker
WHERE salary NOT IN (SELECT MAX(salary) FROM worker);
```

Explanation:

Selects the max salary excluding the highest (giving the second highest).

Q-37. Write an SQL query to show one row twice in results from a table.

```
SELECT * FROM worker
UNION ALL
SELECT * FROM worker
ORDER BY worker_id;
```

Explanation:

Duplicates every row by selecting the table twice with `UNION ALL`.

Q-38. Write an SQL query to list worker_id who does not get bonus.

```
SELECT worker_id FROM worker
WHERE worker_id NOT IN (SELECT worker_ref_id FROM bonus);
```

Explanation:

Finds workers who are not referenced in the `bonus` table.

Q-39. Write an SQL query to fetch the first 50% records from a table.

```
SELECT * FROM worker
WHERE worker_id <= (SELECT COUNT(worker_id)/2 FROM worker);
```

Explanation:

Fetches rows up to half the total count based on `worker_id`.

Q-40. Write an SQL query to fetch the departments that have less than 4 people in it.

```
SELECT department, COUNT(department) AS depCount
FROM worker
GROUP BY department
HAVING depCount < 4;
```

Explanation:

Filters departments with fewer than 4 workers.

Q-41. Write an SQL query to show all departments along with the number of people in there.

```
SELECT department, COUNT(department) AS depCount
FROM worker
GROUP BY department;
```

Explanation:

Counts workers per department.

Q-42. Write an SQL query to show the last record from a table.

```
SELECT * FROM worker WHERE worker_id = (SELECT MAX(worker_id) FRO
M worker);
```

Explanation:

Selects the record with the highest `worker_id`.

Q-43. Write an SQL query to fetch the first row of a table.

```
SELECT * FROM worker WHERE worker_id = (SELECT MIN(worker_id) FRO
M worker);
```

Explanation:

Selects the record with the lowest `worker_id`.

Q-44. Write an SQL query to fetch the last five records from a table.

```
(SELECT * FROM worker ORDER BY worker_id DESC LIMIT 5) ORDER BY w
orker_id
```

Explanation:

Gets last 5 records by descending order, then orders them ascending for display.

Q-45. Write an SQL query to print the name of employees having the highest salary in each department.

```
SELECT w.department, w.first_name, w.salary
FROM (
  SELECT MAX(salary) AS maxsal, department
  FROM worker
  GROUP BY department
) temp
INNER JOIN worker w ON temp.department = w.department AND temp.max
sal = w.salary
```

Explanation:

Finds highest salary per department and lists workers earning that salary.

Q-46. Write an SQL query to fetch three max salaries from a table using co-related subquery.

```
SELECT DISTINCT salary FROM worker w1
WHERE 3 >= (
  SELECT COUNT(DISTINCT salary) FROM worker w2 WHERE w1.salary <=
w2.salary
)
ORDER BY w1.salary DESC;
```

Explanation:

Fetches top 3 distinct salaries using correlated subquery counting salaries greater or equal.

Q-47. Write an SQL query to fetch three min salaries from a table using co-related subquery.

```
SELECT DISTINCT salary FROM worker w1
WHERE 3 >= (
```

```
  SELECT COUNT(DISTINCT salary) FROM worker w2 WHERE w1.salary >=
w2.salary
)
ORDER BY w1.salary ASC;
```

Explanation:

Fetches bottom 3 distinct salaries similarly but counting smaller or equal salaries.

Q-48. Write an SQL query to fetch nth max salaries from a table.

```
SELECT DISTINCT salary FROM worker w1
WHERE n >= (
  SELECT COUNT(DISTINCT salary) FROM worker w2 WHERE w1.salary <=
w2.salary
)
ORDER BY w1.salary DESC;
```

Explanation:

Generalizes to fetch the top-n salaries.

Q-49. Write an SQL query to fetch departments along with the total salaries paid for each of them.

```
SELECT department, SUM(salary) AS depSal FROM worker GROUP BY dep
artment ORDER BY depSal DESC;
```

Explanation:

Aggregates total salary by department, ordered from highest to lowest total.

Q-50. Write an SQL query to fetch the names of workers who earn the highest salary.

```
SELECT first_name, salary FROM worker WHERE salary = (SELECT MAX(sa
lary) FROM worker);
```

Explanation:

Finds workers earning the maximum salary.